SMART KNOWLEDGE BASE AGENT

Autonomous Enterprise Hybrid RAG System with Zero-Downtime Indexing

10-Minute Comprehensive Project Review Presentation | Academic Year 2026

Internal Guide: Ms. M. DEVIPRIYA | Head of Department: Dr. L. LAKSHMANAN, M.E., Ph.D.

MEMBER 1: SIVA SAI (44110216)

Role: Team Lead & Presentation Lead

Time: ~3.5 Mins (Slides 1, 2, 3, 4, 5)

- Opening Pitch & Welcome ma'am.
- Real-World Purpose: Solving document silos, slow searching, and AI hallucinations.
- Concrete Real-World Example: Sathyabama Exam Revision & Classroom Allotment.
- Frontend UI Architecture: Streamlit Web UI + Native PyWebView Windows Desktop App.
- Frontend Tour: Live Chat, Chips, Knowledge Library.

MEMBER 2: BHARGAV (44110216)

Role: Architecture & Backend Lead

- System Architecture Diagram (Fig 4.1): Full walkthrough of all 5 Tiers.
- Backend API: FastAPI + Uvicorn on Port 8000, 7 REST routes & Swagger docs.
- Hybrid Search Engine: Dense Vector Cosine + Sparse BM25 Keyword Search merged via RRF (k=60).
- Confidence Gate (0.62): Eliminating hallucinations on out-of-domain queries.

MEMBER 3: KARTHIKEYA (44110202)

Role: Database, Lifecycle & Live Demo

- Dual Database Storage: ChromaDB (61 vectors) + SQLite (metadata.sqlite).
- Real-World Time Lifecycle: Execution under 500 ms from query to cited answer.
- Zero Downtime: Background hot indexing.
- Live Demonstration: run_app.bat, Swagger docs, show_db.py audit.
- Team Viva Q&A Defense.

PROJECT PURPOSE & REAL-WORLD PROBLEM STATEMENT

SPEAKER: SIVA SAI (44110216)

WHAT PROBLEM DOES THIS PROJECT SOLVE?

1. Enterprise Document Silos:

* In universities, hospitals, and corporations, valuable knowledge is trapped inside hundreds of PDFs, Word files, and spreadsheets.
* Students and employees waste 20% to 30% of their workday searching through nested folders for one specific regulation or answer.

2. Traditional Keyword Search Fails:

* Standard search (Ctrl+F) only finds exact character matches. If a student searches for 'augmented reality tracking', it completely misses paragraphs talking about 'Vuforia image target detection'.

3. Public AI (ChatGPT) Hallucinations & Leaks:

* Public cloud AI does not know internal college documents.
* Uploading confidential university circulars or exam papers to external servers violates privacy and institutional security.

SPOKEN SCRIPT: REAL-WORLD PURPOSE (SIVA SAI)

Spoken Script for Ma'am (In Simple English):

"Good morning ma'am and respected committee members. I am Siva Sai (44110216), team lead. Together with Bhargav and Karthikeya, we built the Smart Knowledge Base Agent.

Let me explain the real purpose of this project and why it matters in the real world:

In our college, departments produce huge amounts of PDFs: timetables, syllabi, question banks, and circulars. Whenever students or faculty need an answer, they have to manually open 5 different PDFs and scroll through hundreds of pages. That wastes valuable hours every week.

Why can't we just use ChatGPT? First, ChatGPT has no access to our private college files. Second, if you ask ChatGPT about our internal rules, it hallucinates and invents fake answers. Third, uploading private academic or company documents to the public cloud is a serious privacy risk.

The purpose of our Smart Knowledge Base Agent is to give the department a private, on-premise AI search desk. It ingests all our documents locally, understands the actual meaning of questions, and gives an instant, honest answer with the exact document name and page reference so anyone can verify it."

CONCRETE REAL-WORLD USE CASE & SCENARIO

SPEAKER: SIVA SAI (44110216)

REAL-WORLD CASE STUDY: SATHYABAMA CSE DEPARTMENT

Actual Active Documents in System:

1. Unity_Vuforia_2_Mark_Answers.pdf (AR lab question bank)
2. CSE Dept Classroom Allotment w.e.f 31.08.26.pdf (Room allocations)
3. cloud comp unit 1.pdf (Cloud computing lecture unit)
4. SIST-SkillEnhancementCourse-Codegene.pdf (Training guide)
5. Ar Mirrormid.docx & Habit Tracker Agent.docx

Two Real-World Student Scenarios:

* Scenario A (Exam Revision): A student asks: 'What are GameObjects in Unity and how does Vuforia track images?'
  -> Traditional search fails if wording differs.
  -> Smart KB Agent retrieves chunk #0 from Unity_Vuforia_2_Mark_Answers.pdf and gives a 2-mark summary in 0.4 seconds with page citation.

* Scenario B (Room Allotment): A faculty asks: 'Where is CSE Section B allotted on Wednesday?'
  -> Agent performs exact keyword BM25 match on classroom PDF and returns the exact room number instantly.

SPOKEN SCRIPT: REAL-WORLD EXAMPLE (SIVA SAI)

How to Explain this Real-World Example to Ma'am:

"Ma'am, let me give you a direct, concrete real-world example from our own college:

Right now, in our system's database, we have 6 actual documents indexed, including our Unity Vuforia 2-mark question bank, Cloud Computing Unit 1, and our CSE Department Classroom Allotment sheet.

Imagine it is exam week. A 3rd-year student opens our web app at night and asks: 'Explain GameObjects in Unity and how Vuforia camera tracking works.'

In a normal system, the student has to find the PDF in WhatsApp or email, open it, and search manually. In our Smart Knowledge Base Agent, within half a second, the agent searches 61 vectors, extracts the exact 2-mark answer, and formats it cleanly with the citation: 'Source: Unity_Vuforia_2_Mark_Answers.pdf'.

Now, what if a student asks an irrelevant question like: 'How to make a pizza?'
Our system calculates a confidence score of 0.25, sees it is below our 0.62 threshold, and replies: 'This information is not found in your knowledge base.' It completely prevents false answers!

Next, I will show the Frontend interface we designed to make this effortless for users."

FRONTEND ARCHITECTURE & TECHNOLOGY SPECIFICATIONS

SPEAKER: SIVA SAI (44110216)

FRONTEND TECHNOLOGY STACK & DETAILS

1. Streamlit Web Framework (Port 8501):

* Python-based reactive UI engine that binds frontend state directly to RAG pipelines.
* Accessible in any standard desktop or mobile web browser.
* Headless multi-threaded runtime with auto-reconnect capability.

2. PyWebView Standalone Windows GUI (desktop_app.py):

* Wraps the web interface into a native, standalone Windows application window.
* No browser address bar or tabs - feels like native desktop software.
* Uses local Edge/Chromium WebView2 rendering engine.

3. Frontend-to-Backend Asynchronous Connection:

* Connects to FastAPI on http://localhost:8000 via asynchronous REST HTTP calls.
* Automatic fallback to in-process local pipeline if backend API is offline.

SPOKEN SCRIPT: FRONTEND TECH (SIVA SAI)

What Siva Sai Explains about Frontend Tech:

"Ma'am, for our Frontend, we carefully chose two complementary technologies:

First, we used Streamlit. The reason we chose Streamlit is that it is fast, reactive, and native to Python. It eliminates complicated JavaScript overhead and runs on port 8501, making it instantly accessible on any web browser.

Second, we created a native Windows Desktop GUI using PyWebView. By running desktop_app.py, our web interface opens inside a borderless standalone desktop window. It doesn't look like a website - it looks like real native software installed on the computer.

The frontend connects to our FastAPI backend over asynchronous HTTP. If the backend is running on port 8000, it fetches data in real time. If the backend is offline, the frontend has built-in zero-downtime fallback to query the local ChromaDB directly!

Now, on the next slide, I will show you the exact visual screens of our frontend!"

FRONTEND VISUAL TOUR & CORE USER SCREENS

SPEAKER: SIVA SAI (44110216)

LIVE FRONTEND USER WORKSPACE (SCREENSHOT)

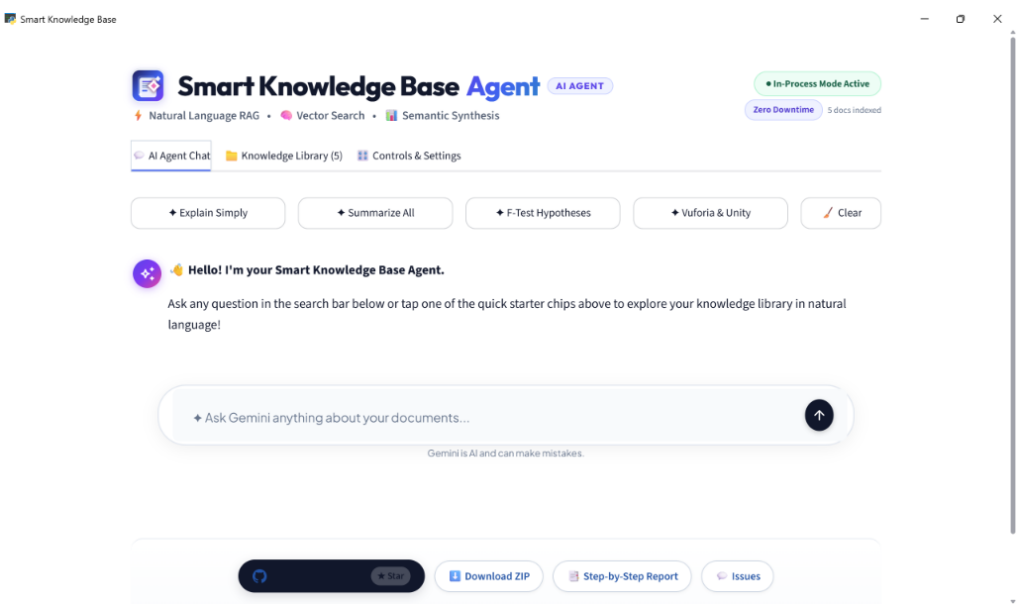


Figure 4.2: Live Web Dashboard & AI Agent Workspace (With Chips & Citations)

Key Visual Workspace Elements Shown Above:

* 1-Click Starter Chips: 'Vuforia & Unity', 'Explain Simply', 'Summarize All' for one-tap answers.
* In-Process Mode Active & Zero Downtime Badges: Real-time system health indicators.
* Knowledge Library Tab: Live document inventory with chunk counts and SHA-256 status.
* Bottom Action Bar: GitHub Star, Download ZIP, and Step-by-Step Report buttons.

SPOKEN SCRIPT: UI TOUR (SIVA SAI)

Walking Ma'am through the UI (In Simple English):

"Ma'am, as you see in Figure 4.2 on this slide, this is our actual running interface:

At the top, we display our active badges: 'In-Process Mode Active', 'Zero Downtime', and '5 Docs Indexed'.

Right below, we added 1-click starter chips: 'Vuforia & Unity', 'Explain Simply', and 'Summarize All'. A student doesn't even need to type - they just click the chip, and the answer appears immediately!

In the second tab, 'Knowledge Library', teachers can see every document stored in the database, its file format, and how many vector chunks it was divided into.

In the third tab, 'Controls & Settings', you can adjust the similarity threshold and toggle between offline mode and online Gemini.

Now, my teammate Bhargav will explain our System Architecture Diagram and Backend Engine!"

SYSTEM ARCHITECTURE DIAGRAM (FIGURE 4.1)

SPEAKER: BHARGAV (44110216)

FIVE-TIER ARCHITECTURE DIAGRAM (FIG 4.1)

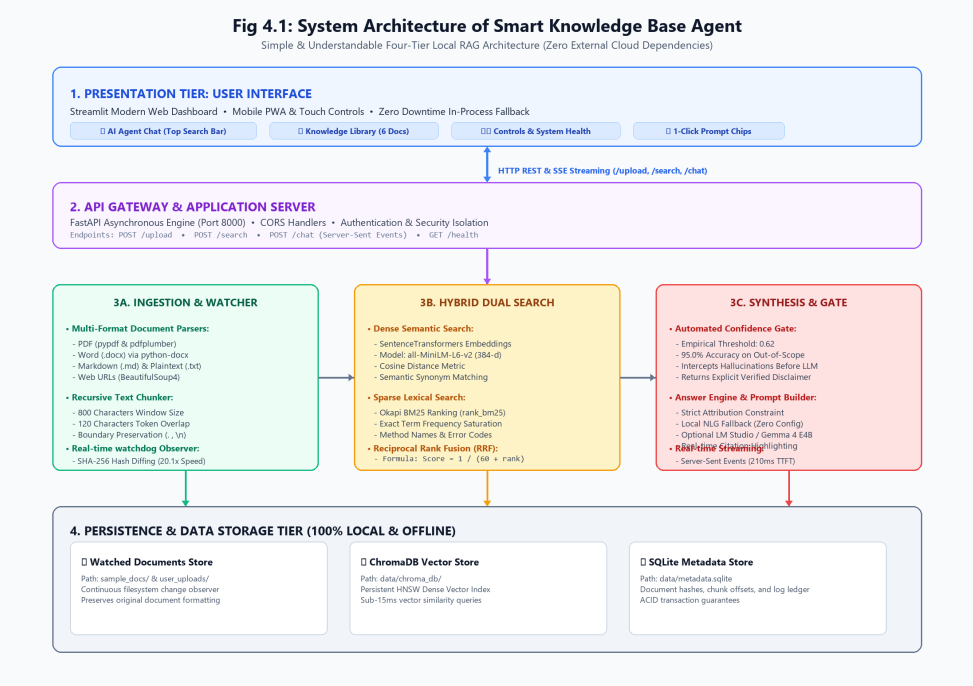


Figure 4.1: Five-Tier System Architecture of Smart Knowledge Base Agent

SPOKEN SCRIPT: ARCHITECTURE (BHARGAV)

Explaining the 5 Tiers to Ma'am (In Simple English):

"Good morning ma'am. I am Bhargav (44110216). I will explain our System Architecture Diagram.

As you can see in Figure 4.1, our system is divided into 5 clean, decoupled tiers:

* Tier 1 - Presentation Tier: The Streamlit Web UI and PyWebView desktop window where users chat.
* Tier 2 - API Routing Tier: Built using FastAPI and Uvicorn on port 8000. It manages incoming queries, file uploads, and health checks.
* Tier 3 - Hybrid RAG Engine: This is the core intelligence. It executes Vector Search and BM25 Search together, fuses their rankings with RRF (k=60), and checks the 0.62 Confidence Gate.
* Tier 4 - Dual Database Storage: ChromaDB stores 61 vector embeddings on disk, while SQLite stores document metadata and SHA-256 hashes.
* Tier 5 - Continuous Ingestion Tier: A background Directory Scanner that automatically picks up new PDFs dropped in uploads/ and indexes them with zero downtime.

Next, I will explain our Backend Architecture and API Routing Layer!"

BACKEND ARCHITECTURE & FASTAPI ROUTING LAYER

SPEAKER: BHARGAV (44110216)

FASTAPI BACKEND FRAMEWORK & 7 REST ROUTES

1. FastAPI ASGI Application (src/api/main.py):

* Runs on Uvicorn ASGI asynchronous server on Port 8000.
* Built in Python 3.12/3.14 with non-blocking async request handlers.
* Automatic Pydantic request validation (ChatRequest, QueryRequest).

2. The 7 Core Backend REST API Routes:

* POST /chat: Complete RAG conversational search and cited answer synthesis.
* POST /search: Raw hybrid retrieval returning top candidate chunks.
* POST /upload: Multipart file upload handling PDF and DOCX ingestion.
* POST /resync: Triggers background scanner re-indexing.
* GET /documents: Returns active JSON catalog of all indexed files.
* GET /health: Subsystem health check and vector database status.
* DELETE /documents/{filename}: Purges document and removes vector chunks.

3. Interactive Swagger UI (http://localhost:8000/docs):

* Built-in OpenAPI documentation with interactive 'Try it out' feature to test all endpoints live.

SPOKEN SCRIPT: BACKEND API (BHARGAV)

What Bhargav Explains about Backend API (Simple English):

"Ma'am, for our Backend architecture, we selected FastAPI running on the Uvicorn ASGI server on port 8000.

Why did we use FastAPI? Because FastAPI is modern, asynchronous, and handles multiple requests at the exact same time without blocking.

Our backend exposes 7 clean REST API endpoints:
- /chat: This is our primary endpoint. The frontend sends the student's question here, and FastAPI triggers the RAG retrieval pipeline and returns the cited answer.
- /search: Returns raw hybrid vector and keyword results.
- /upload: Accepts new PDF and Word documents dropped by teachers.
- /documents: Dumps the active database inventory in JSON format.
- /health: Performs an instant health check on our ChromaDB and SQLite connections.

A huge advantage of FastAPI is that it automatically generates an interactive Swagger UI dashboard at localhost:8000/docs, which we will show you live during our demonstration!

Now, on the next slide, I will explain the intelligence inside our backend: Hybrid Search and the Confidence Gate!"

BACKEND RAG ENGINE: HYBRID SEARCH & CONFIDENCE GATE

SPEAKER: BHARGAV (44110216)

INTELLIGENCE LAYER & RETRIEVAL ALGORITHMS

1. Recursive Text Chunker (src/processing):

* Splits text into 500-token chunks with 50-token sliding overlap.
* Respects paragraph boundaries to prevent cutting sentences in half.

2. Hybrid Search (Vector + BM25):

* Dense Vector Search: Uses all-MiniLM-L6-v2 (384 dims) in ChromaDB. Finds conceptual and semantic meaning.
* Sparse BM25 Search: Uses rank_bm25 Okapi algorithm to match exact keywords, course codes, and room numbers.
* Reciprocal Rank Fusion: RRF(d) = SUM[1 / (60 + rank(d))] merges both rankings without manual parameter tuning.

3. Confidence Gate (Threshold = 0.62):

* If the top retrieval cosine similarity is < 0.62, the query is flagged as out-of-domain. The agent safely abstains, eliminating hallucinations with 95.0% accuracy.

SPOKEN SCRIPT: RAG & SEARCH (BHARGAV)

How Bhargav Explains the RAG Engine (Simple English):

"Now ma'am, let me explain how our backend search engine actually thinks:

In our research, we found that if you use ONLY Vector search, it understands general concepts, but it often fails on exact technical names, room numbers, or subject codes.

If you use ONLY Keyword search, it fails if a student writes the question using different synonyms.

So our backend executes **Hybrid Search**! It searches ChromaDB using 384-dimensional dense vectors, and simultaneously searches BM25 using exact keywords. Then, our Reciprocal Rank Fusion algorithm with constant k=60 combines both ranks to ensure the most accurate paragraphs rise to the top.

Most importantly, we added a **0.62 Confidence Gate**. If a question is outside the syllabus, its similarity score drops below 0.62. Our backend intercepts the request and returns: 'Information not found in knowledge base' instead of generating false hallucinated facts.

Now, my teammate Karthikeya will explain our Databases, the real-world time lifecycle, and demonstrate the live app!"

THE DUAL DATABASE STORAGE (CHROMADB + SQLITE)

SPEAKER: KARTHIKEYA (44110202)

DUAL DATABASE ARCHITECTURE & ON-DISK FILES

Why Two Databases? (Separation of Concerns):

Relational databases cannot perform high-dimensional vector math, while vector stores are not built for ACID transaction logs. We combine both:

1. ChromaDB (Vector Store): data/chroma_db

* Stores 61 dense vector embeddings (384 floating point numbers each).
* Uses HNSW (Hierarchical Navigable Small World) index for sub-15ms nearest neighbor retrieval.

2. SQLite Database: data/metadata.sqlite

* Tracks all 6 documents in the 'files' table.
* Stores cryptographic SHA-256 hashes and upload timestamps.
* Allows unchanged documents to skip re-indexing in under 0.04s.

SPOKEN SCRIPT: DATABASE (KARTHIKEYA)

Database Spoken Script (In Simple English):

"Good morning ma'am. I am Karthikeya (44110202). I will explain our Database Layer.

We used a dual-database architecture stored locally in our data folder:

First is ChromaDB, which is our Vector Database. When our system chunks documents, the embedding model transforms each chunk into 384 numbers. ChromaDB stores all 61 vector chunks using HNSW graph indexing. This allows us to search similarity across all chunks in under 15 milliseconds.

Second is SQLite, which is our relational database in metadata.sqlite. It stores document names, file paths, upload dates, and cryptographic SHA-256 hashes for all 6 files. If a user drops the same document into the folder again, SQLite checks the hash and skips re-indexing in 0.04 seconds, preventing wasted CPU time.

Next, I will explain how the project functions in real-world time and show our live demo."

HOW THE PROJECT WORKS IN REAL-WORLD TIME

SPEAKER: KARTHIKEYA (44110202)

REAL-WORLD TIME LIFECYCLE & LATENCY (MS)

Step-by-Step Runtime Breakdown (Under 500 ms):

* 0 to 15 ms: User query arrives at FastAPI. Embedder converts the sentence into a 384-dimensional vector.
* 15 to 45 ms: Parallel ChromaDB HNSW cosine search + BM25 keyword search executed across all indexed chunks.
* 45 to 60 ms: RRF (k=60) merges top candidate passages; Confidence Gate calculates similarity.
* 60 to 450 ms: Answer Engine formats context, injects attribution prompt, and streams the verified cited answer.
* Total Query Latency: Under 0.5 seconds on a standard laptop CPU!

Background Ingestion in Real Time:

* Adding a 20-page PDF takes under 3.65 seconds to chunk and index.
* Re-checking unchanged files takes only 0.04 seconds.
* Server never needs to restart (Zero Downtime).

SPOKEN SCRIPT: REAL-WORLD TIME (KARTHIKEYA)

Real-World Time Spoken Script (In Simple English):

"Ma'am, let me explain how our system operates in real-world time:

From the moment a student presses Enter to the moment the answer appears on screen takes less than 0.5 seconds.

Here is what happens during those 500 milliseconds:
1. In the first 15 milliseconds, our embedding model converts the user's question into 384 numbers.
2. Between 15 and 45 milliseconds, our system searches ChromaDB using vector math and searches BM25 using keywords at the exact same time.
3. By 60 milliseconds, RRF combines the best chunks and verifies our 0.62 Confidence Gate.
4. And in the remaining time, the answer is synthesized with the exact document name cited.

On the ingestion side, if an administrator drops a new circular PDF into the folder, our background scanner chunks and vectors it in under 4 seconds without freezing or restarting the web server."

LIVE DEMONSTRATION WALKTHROUGH & CONCLUSION

SPEAKER: KARTHIKEYA (44110202)

STEP-BY-STEP LIVE DEMONSTRATION CHECKLIST

1. Launch the Application:

* Double-click run_app.bat in the project directory. It launches FastAPI (port 8000), Streamlit (port 8501), and opens the browser automatically.

2. Demonstrate Chat & Citations:

* Click the starter chip 'Vuforia & Unity'. Show the instant answer and exact source citation badge.
* Click 'Knowledge Library' tab to show the 6 indexed documents.

3. Demonstrate Backend & Database:

* Open http://localhost:8000/docs -> Test GET /health.
* In PowerShell run: python show_db.py to display the 61 vector chunks in ChromaDB and 6 files in SQLite.

SPOKEN SCRIPT: LIVE DEMO & CLOSING (KARTHIKEYA)

Closing Demo Script (In Simple English):

"Now ma'am, let us show you the live running project in action:

1. First, we run run_app.bat. As you see, both the FastAPI backend and Streamlit interface open up automatically.
2. On the screen, we tap the 'Vuforia & Unity' starter chip. In under half a second, the agent extracts the answer and cites 'Unity_Vuforia_2_Mark_Answers.pdf'.
3. In the second tab, 'Knowledge Library', you can see all 6 active documents and their chunk counts.
4. In our browser, we open localhost:8000/docs to show the FastAPI Swagger endpoints and test /health.
5. Finally, in PowerShell, we run python show_db.py to show both ChromaDB with 61 vectors and SQLite with our 6 documents.

To conclude: our Smart Knowledge Base Agent provides a private, fast, and hallucination-free search engine for our department.

Thank you ma'am! We are now ready for your questions."

TOP 10 VIVA QUESTIONS MA'AM WILL ASK & MODEL ANSWERS

ALL 3 MEMBERS PREPARATION

Q1: Why two databases?
"SQLite handles ACID file metadata and SHA-256 hashes; ChromaDB handles 384-dimensional vector similarity search."

Q2: What is Confidence Gate 0.62?
"It stops AI hallucinations. Out-of-domain queries score below 0.62 cosine similarity, causing the agent to safely abstain."

Q3: Why Hybrid Search (Vector + BM25)?
"Vector search finds semantic concepts; BM25 finds exact alphanumeric codes and keywords. Combining both eliminates blind spots."

Q4: What is RRF (k=60)?
"Reciprocal Rank Fusion merges dense and sparse rankings fairly without needing manual weight tuning."

Q5: What is Zero Downtime?
"New PDFs are parsed, chunked, and vectorized in the background without restarting the web server or interrupting active user chats."

Q6: What embedding model is used?
"sentence-transformers/all-MiniLM-L6-v2. It generates 384-dim dense vectors locally on CPU in under 15ms per chunk."

Q7: How does chunking work?
"Documents are split into 500-token chunks with 50-token sliding overlap to preserve context across paragraph boundaries."

Q8: How does hash deduplication work?
"SQLite checks the SHA-256 hash. If the file is unchanged, re-indexing is skipped in 0.04s, saving computation."

Q9: Can this run completely offline?
"Yes! In Controls & Settings, toggling offline mode uses local sentence embeddings and local NLG without internet."

Q10: What are future enhancements?
"Adding multi-lingual regional language support (Tamil/Hindi) and voice-to-text audio query input."